ecbench run report: Callgrind instruction counts of the `rho.negation` and `ic.pipeline` children, before and after the alias-table, walk and registry-index changes

Run start: 2026-11-01T01:30Z (UTC). **Study:** `research/ecbench_child_walk_ir_20261101/` (README with the result tables, `data/`, `inputs/`, `plot.py`). **Class:** engineering measurement of the harness's own instruction counts. No ECDLP cost changes: every child's report (phases, GAE, counters, recovered logarithm) is byte-identical across the binaries; only wall-clock and placement fields differ.

What ran

- **Command.** For each exported child input, `valgrind --tool=callgrind --cache-sim=no --branch-sim=no <ecbench> exec < input.json`. The instruction count is Callgrind's `Collected` total for the whole child process (spawn to exit, every dump part); the solve-only count is `ecbench callgrind-ir` over the marker parts the child dumps under `ECBENCH_CALLGRIND_SOLVE=1`, i.e. the same split `ecbench audit-ir` prints. The 20-bit rows were produced by `ecbench audit-ir` directly.
- **Inputs.** `inputs/spec-p32-profile.json` (two public 32-bit targets, arms `rho-neg` and `inc` at $N = 1024$); the exported child inputs for seq 0 (`rho.negation`) and seq 1 (`ic.pipeline`) are identical under both binaries (`cmp`). The 20-bit inputs are `inputs/spec-p20-profile.json`, seq 0–2.
- **Binaries.** baseline: `main` 8a208fa570 (2026-10-31), rebuilt for the 32-bit rows from the same two source files as the earlier baseline build (sha256 b661f953d79dccd8…; the earlier build b8775974f8d8d876… produced the 20-bit rows); it reproduces the 20-bit `rho.negation` child within 0.03% (prework 206,888,568 vs 206,896,333 Ir, solve 1,295,929 vs 1,295,567 Ir; the baseline's alias-table `HashMap` is randomly seeded, so its count moves by about 10^4 Ir between runs). B: baseline + `curve_id.rs` + `ic_boundary.rs` (sha256 0eff84e84d22b1a6…). C: B + `workload.rs` + `build.rs`, the registry index (sha256 80f5ecb7079f01e2…).
- **Host.** Shared 4-vCPU cloud VM, Intel Xeon @ 2.10 GHz, no hardware PMU; isolation L0. valgrind 3.22.0, rustc 1.97.0. Instruction counts are deterministic on this host (two runs of the B rho child: 69,591,262 and 69,617,226 Ir, the second with the marker dumps); wall times are not reported from these runs.
- **Completed.** 6 of 6 requested 32-bit whole-process runs (two children × three binaries) plus a marker run of the rho child under each binary; 10 of 10 20-bit `audit-ir` runs (baseline seq 0–2, A seq 0–1, B seq 0–1, C seq 0–2). No failures. A first 32-bit `ic.pipeline` attempt with per-marker dump files was stopped: the child dumps a part at every relation marker (hundreds of thousands of files at this size), so the whole-process runs discard the dump data (`--combine-dumps=yes --callgrind-out-file=/dev/null`) and read the `Collected` total; this is why the IC row has no solve-only count.

Measured values (Callgrind Ir)

child	binary	whole process	before the solve marker	solve	GAE	solve Ir / GAE
`rho.negation`, 32-bit (seq 0)	baseline	274,792,709	242,782,512	31,726,911	34,156	928.9
	B	69,591,262	39,232,509	**30,093,469**	34,156	**881.1**
	C	**32,838,412**	**2,475,446**	30,092,508	34,156	881.0
`ic.pipeline` $N = 1024$, 32-bit (seq 1)	baseline	1,147,910,545	–	–	1,446,809	–
	B	944,194,507	–	–	1,446,809	–
	C	**907,443,128**	–	–	1,446,809	–
`rho.negation`, 20-bit (seq 0)	baseline	–	206,896,333	1,295,567	1,706	759.4
	B	–	3,331,282	1,222,380	1,706	716.5
	C	–	2,437,871	1,222,603	1,706	716.7
`ic.pipeline` $N = 32$, 20-bit (seq 1)	baseline	–	206,928,755	14,889,758	14,101	1,055.9
	B	–	3,362,189	14,887,062	14,101	1,055.7
	C	–	2,470,003	14,888,160	14,101	1,055.8

Derived columns: "solve Ir / GAE" is the solve count divided by the child's reported `total_gae`. Whole-process differences: baseline → B 205,201,447 Ir (rho child) and 203,716,038 Ir (IC child), against the 203.5M-Ir alias-table parse the 20-bit rows isolate (206.9M − 3.3M before the solve marker); B → C 36,752,850 Ir (rho) and 36,751,379 Ir (IC), the registry scan (`callgrind_annotate` of B's 32-bit pre-solve part: 37.0M of 39.2M in one `str::find` from `Instance::facts`). What remains before the solve marker under C (2.5M at 32 bits, 2.4M at 20 bits) is process start, the argument parser, the entry's own parse and the identity.

- **Before the solve marker.** The alias-table parse (A) is gone from every child, −203.5M Ir at both sizes, and the registry scan (C) with it: −36.8M at 32 bits, −0.9M at 20 bits, where the curve's entry is near the start of the document.
- **`rho.negation` solve.** −5.1% at 32 bits (928.9 ⟶ 881.1 Ir/GAE), −5.7% at 20 bits; the walk's set-up and verification are a smaller share at 32 bits, so this is nearer the steady-state per-step saving.
- **`ic.pipeline`.** Its relation walk is unchanged (20-bit solve within 0.02%); at 32 bits the whole-process difference is the parse alone.
- **Identity.** `verify --replay-all --exit-code` reproduced 36/36 (20-bit) and 4/4 (32-bit) session records under every binary; the 32-bit child outputs under B and C differ from the baseline's only in `method_call_wall_ns`, `solve_wall_ns`, `phases_ns`, `schedstat`, `solve_obs` and `placement` fields.

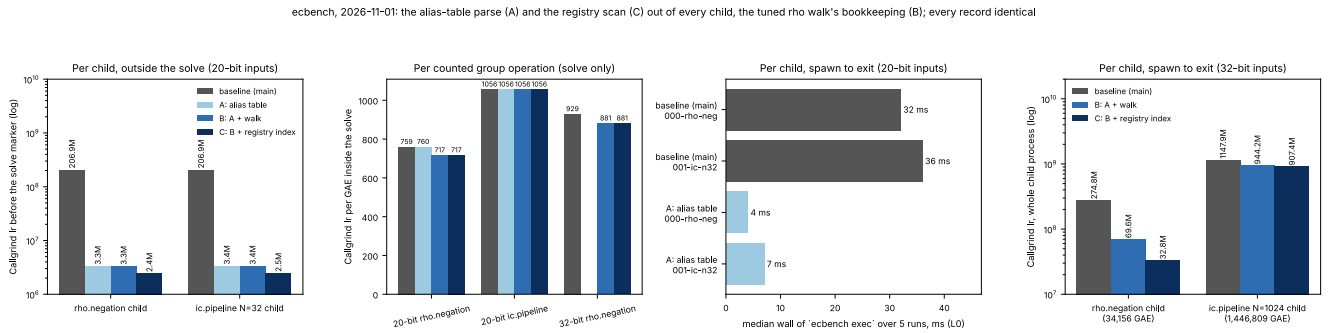


Figure 1: Recorded values only (`data/audit_ir_20bit.tsv`, `data/audit_ir_32bit.tsv`, `data/child_wall_20bit.tsv`): instructions before the solve marker per child (20-bit), solve Ir per GAE (20- and 32-bit), the 20-bit child wall medians (5 runs, L0, descriptive), and the whole-process 32-bit counts. Sessions ECBS1haae438ff6051 (20-bit) and ECBS1h8f9347fe1d3e (32-bit).

Raw records

`data/audit_ir_20bit.tsv`, `data/audit_ir_32bit.tsv`, `data/child_wall_20bit.tsv`; the exported child inputs in `inputs/`; the two scratch sessions are replay-verified but not committed (they were run with `--allow-busy` on the shared host and carry no isolation claim). Interpretation and the limits of the conclusion are in the study README.